

# Немного ещё о CTE

- CTE (Common Table Expression) – элемент стандарта SQL. Это как бы временная таблица или view, существующая в рамках одного запроса.
- Цель – разбить сложные запросы на более читаемые части.

```
WITH alias1 AS (SELECT query1),  
      alias2 AS (SELECT query2)  
SELECT * FROM alias1, ...
```

# Подробности CTE

- В Postgres возможны рекурсивные CTE
- Возможны также пишущие CTE
- Различные СУБД по разному воспринимают CTE. Часто это View.
- В Postgres это VIEW либо временная таблица. CTE м.б. Материализовано или Нет.
- Материализованное - барьер для оптимизатора

# Ещё СТЕ

- Пишущие СТЕs исполняются всегда
- Неиспользуемые СТЕ не исполняются

```
WITH          yy  AS MATERIALIZED ( SELECT * FROM cte WHERE y > 1 ),
      not_executed AS          ( SELECT * FROM cte ),
      always_executed AS ( INSERT INTO cte VALUES(2,2) RETURNING *)
SELECT FROM yy WHERE x=2;
```

Пусть есть индекс по (x,y)

```
Index Only Scan using cte_i on cte
  Index Cond: ((x = 2) AND (y > 1))
  CTE always_executed
    -> Insert on cte cte_1
        -> Result
```

```
CTE Scan on yy
  Filter: (x = 2)
  CTE yy
    -> Seq Scan on cte
        Filter: (y > 1)
  CTE always_executed
    -> Insert on cte cte_1
        -> Result
```

# Ещё СТЕ

- Пишущие СТЕs исполняются всегда
- Неиспользуемые СТЕ не исполняются

```
WITH          yy  AS MATERIALIZED ( SELECT * FROM cte WHERE y > 1 ),
      not_executed AS          ( SELECT * FROM cte ),
      always_executed AS ( INSERT INTO cte VALUES(2,2) RETURNING *)
SELECT FROM yy WHERE x=2;
```

Пусть есть индекс по (x,y)

```
Index Only Scan using cte_i on cte
  Index Cond: ((x = 2) AND (y > 1))
  CTE always_executed
    -> Insert on cte cte_1
        -> Result
```

```
CTE Scan on yy
  Filter: (x = 2)
  CTE yy
    -> Seq Scan on cte
        Filter: (y > 1)
  CTE always_executed
    -> Insert on cte cte_1
        -> Result
```

# Ещё о материализации CTE

```
WITH cte_name AS [NOT] MATERIALIZED ...
```

- Пишущие запросы в CTE всегда материальны
- Рекурсивные – тоже
- По умолчанию – NOT Materialized (с v12 :)
- Если WITH-запрос используется более одного раза ...
  - Если Materialized – он будет выполнен 1 раз
  - Если нет – более одного (точнее, зависит от оптимизатора)
- Материализация иногда полезна

# Старт из Каламы!

```
WITH RECURSIVE way AS (  
    SELECT DISTINCT arrival_airport, 1 num,  
        ARRAY['CJC',arrival_airport] path  
    FROM routes WHERE departure_airport = 'CJC'  
    UNION ALL  
    SELECT routes.arrival_airport, num + 1,  
        path || ARRAY[routes.arrival_airport]  
    FROM way, routes  
    WHERE routes.departure_airport = way.arrival_airport  
        AND num < 3      -- ограничение рекурсии необходимо  
        AND NOT ( routes.arrival_airport = ANY(path))  
)  
SELECT * FROM way ORDER BY  arrival_airport, num, path;
```

# Вроде хорошо, но...

| arrival_airport | num | path          |
|-----------------|-----|---------------|
| AER             | 2   | {CJC,BOM,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |
| AER             | 2   | {CJC,DME,AER} |

Видно, что есть повторы, и дальше мы для каждого из них повторяем все расчеты — явно лишняя работа.

Больше 5 становится совсем туго. Есть смысл сократить одинаковое.

# Сокращаем перебор

```
WITH RECURSIVE way AS (  
  SELECT DISTINCT arrival_airport, 1 num,  
         ARRAY['CJC',arrival_airport] path  
    FROM routes WHERE departure_airport = 'CJC'  
 UNION ALL  
  SELECT DISTINCT routes.arrival_airport, num + 1,  
         path || ARRAY[routes.arrival_airport]  
    FROM way, routes  
   WHERE routes.departure_airport = way.arrival_airport  
         AND num < 3  
         AND NOT ( routes.arrival_airport = ANY(path) )  
)  
SELECT * FROM way ORDER BY arrival_airport, num, path ;
```



# Ещё сокращаем перебор

```
WITH RECURSIVE
connection AS MATERIALIZED (
    SELECT DISTINCT arrival_airport, departure_airport
    FROM routes
),
way AS (
    SELECT arrival_airport, 1 num,
           ARRAY['CJC', arrival_airport] path
    FROM connection WHERE departure_airport = 'CJC'
    UNION ALL
    SELECT c.arrival_airport, num + 1,
           path || ARRAY[c.arrival_airport]
    FROM way, connection c
    WHERE c.departure_airport = way.arrival_airport
           AND num < 4
           AND NOT ( c.arrival_airport = ANY(path) )
)
SELECT * FROM way ORDER BY arrival_airport, num, path ;
```

# Как растёт количество вариантов

| num | count   |
|-----|---------|
| 1   | 16      |
| 2   | 410     |
| 3   | 10458   |
| 4   | 262947  |
| 5   | 6480331 |

Вопросы:

- как я написал этот запрос ?
- почему он быстрее предыдущего?

# Объединяем эквивалентные маршруты

Как отсортировать массив ?

```
SELECT array_agg(x ORDER BY x)
FROM unnest (path) x;
```

Что, если вместо path будем хранить  
отсортированный path ?

Тогда можно будет сделать DISTINCT, и убрать  
повторы...

| num | count   |
|-----|---------|
| 1   | 16      |
| 2   | 410     |
| 3   | 9757    |
| 4   | 206382  |
| 5   | 3866795 |

# Эвристика «неудачники не нужны»

...

```
UNION ALL
```

```
SELECT c.arrival_airport, num + 1,
```

```
       MIN(path || ARRAY[c.arrival_airport])
```

```
FROM way, connections c
```

```
WHERE c.departure_airport = way.arrival_airport
```

```
AND num < 5
```

```
AND NOT ( c.arrival_airport = ANY(path))
```

```
GROUP BY c.arrival_airport, num
```

```
)
```

```
SELECT * FROM way ORDER BY arrival_airport, num, path;
```

# Внезапное препятствие

...

```
UNION ALL
```

```
SELECT c.arrival_airport, num + 1,
```

```
    MIN(path || ARRAY[c.arrival_airport])
```

```
FROM way, connection c
```

```
WHERE c.departure_airport = way.arrival_airport
```

```
AND num < 5
```

```
AND NOT ( c.arrival_airport = ANY(path))
```

```
GROUP BY c.arrival_airport, num
```

```
)
```

```
SELECT * FROM way ORDER BY arrival_airport, num, path;
```

**ERROR: aggregate functions are not allowed in a recursive query's recursive term**

# Обходной маневр

Заменяем группировку на window function и distinct!

```
...
UNION ALL
  SELECT DISTINCT arrival_airport, num, path FROM (
    SELECT c.arrival_airport, num + 1 AS num,
      MIN(path || ARRAY[c.arrival_airport])
        OVER(PARTITION BY c.arrival_airport, num) AS path
    FROM way, connection c
    WHERE c.departure_airport = way.arrival_airport
    AND num < 100
    AND NOT ( c.arrival_airport = ANY(path))
  ) grouped
)
SELECT * FROM way ORDER BY arrival_airport, num, path;
```

# Получилось!

| arrival_airport | num |                                                                                     |
|-----------------|-----|-------------------------------------------------------------------------------------|
| YYZ             | 72  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YYC             | 72  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YYZ             | 71  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YYZ             | 71  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YVR             | 71  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YOW             | 71  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YVR             | 70  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YYC             | 70  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YYZ             | 70  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YOW             | 70  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YUL             | 70  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YOW             | 69  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YVR             | 69  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YYC             | 69  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YYZ             | 69  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |
| YEG             | 69  | {CJC,ATL,AKL,CAN,BGY,ARN,AER,AMD,BER,BOM,BLR,CCU,COK,CDG,CLT,DEN,DFW,CTS,CKG,CTU,DE |

SELECT MAX(num) FROM way;

А вообще в базе ?

# Другая эвристика!

```
...
UNION ALL
SELECT DISTINCT arrival_airport,num,path FROM (
  SELECT c.arrival_airport, num + 1 AS num,
    ( SELECT array_agg(x ORDER BY x) FROM unnest(
      path || ARRAY[c.arrival_airport]) x
    ) AS path
  FROM way, connection c
  WHERE c.departure_airport = way.arrival_airport
  AND num < 100
  AND NOT ( c.arrival_airport = ANY(path))
  ORDER BY random() LIMIT 1000
) grouped
)
SELECT num,arrival_airport, path FROM way ORDER BY num DESC;
```

Как результат?



# Эвристики

- Могут быть разными, в т.ч. более сложными.
- Например: не заходить в тупиковые узлы, и минимально «сжигать» большие хабы. (Но в этой базе нет тупиков и суперхабов).
- В данном случае и очень простые эвристики дают хороший результат.

# Другое ДЗ

- Превратить дерево в БД (записи типа id, pos, parent, title)  
в дерево в JSON  
(иерархическая структура в виде массива объектов)

# Агрегируем рубрикатор в одно дерево

Задача: Собрать записи узлов рубрикатора в многоуровневое дерево JSON.

```
[{  id: 1, title: 'СУБД', nodes: [  
    { id: 2, title: 'Реляционные СУБД', nodes: [  
        .....  
        и т.д.
```

- Хочется RSTE, но непонятно, как его привинтить:
  - STE умножает количество строк, а нам надо углублять дерево, на выходе получится одна строка.
- Эврика! Нужен агрегат. Он уменьшает количество строк!

# План решения

- 1) Агрегируем таблицу специальным хитрым агрегатом, который ставит каждый узел на свое место в дереве.
- 2) Чтобы этот агрегат был проще, подготовим дерево к агрегации помощью RSTE:
  - для каждого узла определим его место (путь) в результирующем дереве.

# Шаг 1. Вычислим путь в дереве для каждого узла

```
WITH RECURSIVE
  positioned_section AS (
    SELECT *,
      ROW_NUMBER() OVER (PARTITION BY parent ORDER BY id) -1 AS pos
    FROM section
  ),
  t AS (
    SELECT id,title,
      ARRAY['nodes', pos::text] AS path
    FROM positioned_section WHERE parent IS NULL
    UNION ALL
    SELECT s.id, s.title,
      t.path || ARRAY['nodes',pos::text] AS path
    FROM positioned_section s JOIN t ON s.parent = t.id
  )
SELECT row_to_json(t)::jsonb node FROM t ORDER BY path;
```

# Результат шага 1:

node

---

```
{"id": 1, "path": ["nodes", "0"], "title": "СУБД"}  
{"id": 3, "path": ["nodes", "0", "nodes", "0"], "title": "PostgreSQL"}  
{"id": 2, "path": ["nodes", "1"], "title": "ОС"}  
{"id": 4, "path": ["nodes", "1", "nodes", "0"], "title": "Solaris"}  
(4 rows)
```

## Шаг 2. Агрегат, ставящий узлы на места

```
CREATE OR REPLACE FUNCTION tree_agg_f (state jsonb, item jsonb)
  RETURNS jsonb LANGUAGE sql IMMUTABLE STRICT AS $$
  SELECT jsonb_set(
    state,
    ( SELECT array_agg(x)
      FROM jsonb_array_elements_text(item->'path') x
    ),      -- эта конструкция превращает JSON-овский массив в обычный
    jsonb_build_object(
      'id',      item->'id',
      'title',   item->'title',
      'nodes',   '[]'::jsonb
    ),
    true);
$$ ;

CREATE AGGREGATE tree_agg (item jsonb) (
  SFUNC=tree_agg_f, STYPE=jsonb, INITCOND='{"nodes":[]}');
```

# Объединяем 1+2

```
SELECT jsonb_pretty(tree_agg(x.node)) FROM (  
  WITH RECURSIVE  
    positioned_section AS (  
      SELECT *,  
        ROW_NUMBER() OVER (PARTITION BY parent ORDER BY id) -1 AS pos  
      FROM section  
    ),  
    t AS (  
      SELECT id,title,  
        ARRAY[pos] AS path,  
        ARRAY['nodes', pos::text] AS npath  
      FROM positioned_section WHERE parent IS NULL  
      UNION ALL  
      SELECT s.id, s.title, t.path || ARRAY[s.pos::path] AS path,  
        t.npath || ARRAY['nodes',s.pos::text] AS npath  
      FROM positioned_section s JOIN t ON s.parent = t.id  
    )  
  SELECT row_to_json(t)::jsonb node FROM t ORDER BY path  
)  
x;
```



# Другое ДЗ

- Придумать, как обеспечить корректность дерева (отсутствие циклов)